v3.7

TECHNICAL DESIGN DOCUMENT

FACTS.

File and Case Tracking System — Engineering Specification

VERSION	DATE	STATUS	CLASSIFICATION
3.7	April 23, 2025	Final	Internal

Node.js + Express | React 18 + React Query | MySQL 8.0 InnoDB | JWT + RBAC | PDFKit + Puppeteer | Jamiat/Jamaat Scoping

Baaseteen & SHND Organizations
Prepared by FACTS Development Team

Technical Design Document (TDD)

FACTS — File and Case Tracking System

Document Version: 3.7 | **Date:** April 23, 2025 | **Prepared By:** FACTS Development Team | **Status:** Final | **Classification:** Internal

Document Control

ITEM	DETAIL
Project Name	FACTS — File and Case Tracking System
Document Type	Technical Design Document (TDD)
Version	3.7
Related Documents	PRD v1.0, API_SCHEMAS.md, ERD (Mermaid classDiagram), README.md
Organizations	Baaseteen, SHND (Social Help and Development)
Codebase	Monorepo: `/backend` (Node.js), `/frontend` (React), `/database` (MySQL)

Table of Contents

1. Introduction & Purpose
2. System Architecture
3. Technology Stack & Rationale
4. Database Design
5. Core Module Designs
   - 5.1 Authentication & JWT Middleware
   - 5.2 Organizational Scoping Engine
   - 5.3 Case Identification Pipeline
   - 5.4 Case Management & Status Machine
   - 5.5 Counseling Form Architecture
   - 5.6 Workflow & Approval Engine
   - 5.7 SLA Calculation Engine
   - 5.8 Fund Utilization Dynamic Form Builder
   - 5.9 PDF Generation Pipeline
   - 5.10 ITS External API Integration
   - 5.11 Notification Service
   - 5.12 File Upload & Attachment Handler
6. API Design Patterns
7. Frontend Architecture
8. Security Architecture
9. Performance & Scalability
10. Deployment & Infrastructure
11. Error Handling & Logging

1. Introduction & Purpose

1.1 Purpose

This Technical Design Document (TDD) is the authoritative engineering specification for the FACTS system. It translates the product requirements defined in the PRD v1.0 into precise architectural decisions, data flow diagrams, database schema rationale, module-level implementation patterns, security controls, and deployment topology — providing a complete reference for developers, DevOps engineers, and technical reviewers building or maintaining the system.

1.2 Scope

This document covers all backend services, frontend architecture, database design, external integrations, and infrastructure configuration for FACTS v1.0. It does not re-document product-level requirements (refer to PRD v1.0) or reproduce the full API request/response schemas (refer to API_SCHEMAS.md).

1.3 Design Principles

PRINCIPLE	IMPLEMENTATION
Defense in depth	Security enforced at middleware, route, and query layers — never only at UI
Scope isolation by default	Every query filtered by Jamiat/Jamaat scope at the middleware layer; unrestricted access is the exception, not the default
Audit everything	All state transitions (case status, income edits, workflow actions) are append-only logged with actor + timestamp
Fail gracefully	External API failures (ITS) fall back to manual entry; PDF failures surface user-friendly errors; no unhandled Promise rejections
Separation of concerns	Routes handle HTTP; services handle business logic; utilities handle cross-cutting concerns (SLA, notifications, permissions)

2. System Architecture

2.1 High-Level Component Architecture

```
┌──────────────────────────────────────────────────────────┐
│                        CLIENT TIER                        │
│                                                           │
│   React 18 SPA (Vite/CRA build)                           │
│   ├── React Query  → server state + cache invalidation    │
│   ├── React Context → auth token + user profile           │
│   ├── React Hook Form → form state + validation           │
│   └── Axios Instance → baseURL + Authorization header inject │
└──────────────────┬────────────────────────────────────────┘
                   │  HTTPS  (JSON REST)
                   ▼
┌──────────────────────────────────────────────────────────┐
│                       API TIER (Node.js)                  │
│                                                           │
│   Express 4.18  →  index.js (entry point, port 5000)      │
│   ├── Helmet (HTTP security headers)                      │
│   ├── CORS  (FRONTEND_URL only)                           │
│   ├── express-rate-limit  (login endpoint)                │
│   ├── express.json() + express.urlencoded()               │
│   ├── Multer  (multipart/form-data file uploads)          │
│   │                                                       │
│   ├── middleware/auth.js                                  │
│   │   ├── verifyToken()   → JWT decode + user fetch       │
│   │   ├── checkPermission() → resource:action RBAC        │
│   │   └── applyScopeFilter() → Jamiat/Jamaat WHERE injection │
│   │                                                       │
│   ├── routes/ (25 route modules)                          │
│   │   ├── auth.js, users.js, cases.js, applicants.js      │
│   │   ├── counselingForms.js, coverLetters.js             │
│   │   ├── workflowStages.js, notifications.js             │
│   │   ├── fundUtilization.js, fundUtilizationMaster.js    │
│   │   └── [18 more route files]                           │
│   │                                                       │
│   ├── services/                                           │
│   │   ├── emailService.js     (Nodemailer SMTP)           │
│   │   └── notificationService.js  (DB inserts)            │
│   │                                                       │
│   └── utils/                                              │
│       ├── slaCalculator.js   (business day / calendar math) │
│       ├── activeCasePerIts.js (uniqueness guard)          │
│       ├── permissionUtils.js (permission resolution)      │
│       ├── roleUtils.js       (role hierarchy helpers)     │
│       └── quarterlyReportPdfKit.js (PDFKit builder)       │
└──────────────────┬────────────────────────────────────────┘
                   │  mysql2 connection pool (10 conns)
                   ▼
┌──────────────────────────────────────────────────────────┐
│                     DATA TIER  (MySQL 8.0)                │
│   Database: baaseteen_cms                                 │
│   Engine: InnoDB  |  Collation: utf8mb4_unicode_ci        │
│   ~55 tables  |  FK constraints with CASCADE / SET NULL   │
└──────────────────────────────────────────────────────────┘
                   │
            ┌──────┴──────┐
            ▼             ▼
     uploads/ (local FS)    /var/log/facts_error.log
```

2.2 Request Lifecycle

```
Browser → HTTPS Request
  → Nginx (reverse proxy, TLS termination, static file serving)
    → Express app (port 5000)
      → Global middleware (Helmet, CORS, rate-limit, JSON parser)
        → Route match
          → auth.js: verifyToken()
            → JWT.verify(token, JWT_SECRET)
            → DB: SELECT user WHERE id = payload.id AND is_active = 1
            → Attach req.user = { id, role, executive_level, jamiats[], jamaats[] }
          → auth.js: checkPermission(resource, action) [if route requires it]
            → Resolve permissions from user_roles → role_permissions
            → 403 if missing
          → auth.js: applyScopeFilter() [scope-aware routes]
            → Build WHERE clause fragment injected as req.scopeFilter
          → Route Handler
            → Joi input validation
            → Business logic (DB queries via prepared statements)
            → notificationService / emailService (async, non-blocking)
            → JSON response { data } or { error }
      → Error handler middleware → structured log + safe 500 response
```

2.3 Project Directory Structure

```
FACTS/
├── backend/
│   ├── config/database.js          # mysql2 createPool({ connectionLimit:10 })
│   ├── middleware/
│   │   ├── auth.js                 # verifyToken, checkPermission, applyScopeFilter
│   │   └── permissionMiddleware.js # higher-order wrappers
│   ├── routes/                     # 25 Express Router files
│   ├── services/
│   │   ├── emailService.js         # transporter.sendMail() wrapper
│   │   └── notificationService.js  # createNotification(userId, caseId, msg)
│   ├── utils/
│   │   ├── slaCalculator.js
│   │   ├── activeCasePerIts.js
│   │   ├── permissionUtils.js
│   │   ├── roleUtils.js
│   │   ├── quarterlyReportPdfData.js
│   │   └── quarterlyReportPdfKit.js
│   ├── templates/
│   │   └── quarterlyReportPdfTemplate.js  # Puppeteer HTML template
│   ├── scripts/run-migration.js
│   ├── uploads/                    # Multer destination (gitignored)
│   ├── index.js                    # Express bootstrap
│   └── package.json
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── CounselingForm.js   # 7-section tab form
│   │   │   ├── CoverLetterForm.js
│   │   │   ├── FundUtilizationForm.js
│   │   │   ├── PaymentSchedule.js
│   │   │   ├── WelfareChecklistForm.js
│   │   │   ├── SignaturePad.js     # canvas draw + file upload
│   │   │   ├── CaseSLAStatus.js
│   │   │   └── ui/                 # 16 reusable UI primitives
│   │   ├── pages/                  # 31 page components
│   │   ├── contexts/AuthContext.js
│   │   ├── utils/permissionUtils.js  # usePermission() hook
│   │   └── App.js                  # React Router v6 config
│   └── package.json
│
└── database/
    ├── baaseteen_production_database.sql  # full consolidated schema
    └── migrations/                        # incremental SQL files
```

3. Technology Stack & Rationale

LAYER	TECHNOLOGY	VERSION	RATIONALE
Backend Runtime	Node.js + Express.js	v16+ / v4.18	Non-blocking I/O ideal for concurrent API requests; large middleware ecosystem; JSON-native; familiar to JS-fluent team
Database	MySQL 8.0 (InnoDB)	8.0+	ACID compliance; FK constraints for referential integrity; JSON column support for workflow_history and responses; utf8mb4 for multilingual content
ORM / Driver	mysql2 (native promises)	Latest	Prepared statements native; promise-based async/await; connection pooling; no ORM overhead for complex JOIN-heavy queries
Frontend	React 18 (functional + hooks)	18.2	Component reuse across 31 pages; concurrent features for UX; broad ecosystem for complex form management
Server State	React Query (TanStack)	v3.39	Automatic caching, background refetch, optimistic updates — eliminates manual useEffect data-fetching boilerplate
Form Management	React Hook Form	v7.48	Uncontrolled inputs = minimal re-renders for large counseling forms with 50+ fields per section
Styling	Tailwind CSS	v3.4	Utility-first prevents CSS specificity conflicts across 31 pages; consistent design tokens via tailwind.config.js
Authentication	JWT + bcryptjs	jsonwebtoken v9	Stateless tokens eliminate server-side session storage; bcrypt cost factor 10 provides strong password security
Input Validation	Joi	v17.11	Schema-based validation with rich error messages; validates nested objects (e.g. counseling form sections)
PDF Generation	PDFKit + Puppeteer	v0.14 / v24	PDFKit for programmatic quarterly reports; Puppeteer (headless Chrome) for HTML-template-based cover letters with CSS styling
Excel	xlsx library	v0.18	Read .xlsx/.xls for bulk import; write Excel for export templates; no external binary required
File Upload	Multer	v1.4	Streaming multipart parser; memory-safe disk storage; configurable file type and size filters per route
Security	Helmet + CORS + rate-limit	Latest	Helmet sets 11 HTTP security headers; CORS locks to FRONTEND_URL; rate-limit prevents brute force on login
Email	Nodemailer	v6.9	SMTP transport; async fire-and-forget; optional — in-app notifications are primary channel
Drag & Drop	@dnd-kit	Latest	Accessible drag-and-drop for workflow stage reordering and checklist category reordering

4. Database Design

4.1 Conventions & Paradigms

- **Engine:** InnoDB on all tables — required for FK constraint enforcement and row-level locking.
- **Collation:** `utf8mb4_unicode_ci` — supports full Unicode, emoji, and Arabic/Urdu/Gujarati characters in applicant names and addresses.
- **Primary keys:** `INT AUTO_INCREMENT id` on all tables. No UUIDs — simpler JOINs and lighter index storage.
- **Naming:** All tables and columns use `snake_case`. No reserved word collisions; no abbreviations except well-known ones (its, dcm).
- **Audit columns:** `created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP`, `updated_at TIMESTAMP ON UPDATE CURRENT_TIMESTAMP`, `created_by INT FK(users.id)`, `updated_by INT FK(users.id)` — present on all entity tables.
- **Soft deletes:** Configurable entities (workflow_stages, checklist items) use `is_active TINYINT(1)` instead of DELETE — preserves historical case data integrity.
- **JSON columns:** `LONGTEXT` used for JSON storage (workflow_history, responses_json, options_json, associated_statuses) — compatible with MySQL 5.7+ JSON functions while allowing large payloads.

4.2 Entity Group Schema Detail

A. Identity & Access Control

```
users
  id                  INT AUTO_INCREMENT PK
  username            VARCHAR(50) UNIQUE NOT NULL
  full_name           VARCHAR(200) NOT NULL
  email               VARCHAR(100) UNIQUE NOT NULL
  phone               VARCHAR(20)
  its_number          VARCHAR(50)            -- ITS identifier for ITS API sync
  photo               LONGTEXT               -- Base64 encoded; ~100KB practical limit
  password_hash       VARCHAR(255) NOT NULL  -- bcrypt $2b$10$... string
  role                VARCHAR(50)            -- primary/display role (denormalized)
  executive_level     INT FK → executive_levels.level_number
  is_active           TINYINT(1) DEFAULT 1
  jamiat_id           INT FK → jamiat.id    -- legacy single-jamiat; use user_roles for multi
  jamaat_id           INT FK → jamaat.id
  jamiat_ids          TEXT                  -- comma-separated; denormalized cache
  jamaat_ids          TEXT
  created_at / updated_at

user_roles
  id                  INT PK
  user_id             INT FK → users.id
  role_id             INT FK → roles.id
  assigned_by         INT FK → users.id
  assigned_at         TIMESTAMP
  expires_at          TIMESTAMP NULL         -- optional time-bounded role
  is_active           TINYINT(1) DEFAULT 1
  jamiat_ids          TEXT                  -- scope: which jamiats THIS role grants access to
  jamaat_ids          TEXT                  -- scope: which jamaats THIS role fills cases in

roles
  id / name / display_name / description / permissions LONGTEXT / is_active / is_system_role

role_permissions
  id / role_id FK / permission VARCHAR(100) / resource VARCHAR(100) / action VARCHAR(50)

counseling_form_stage_permissions
  id / role_id FK / stage_key / stage_name / can_read / can_update
```

B. Organizational Hierarchy

```
jamiat
  id / jamiat_id VARCHAR(20) / name VARCHAR(200) / is_active
  -- Top-level organizational unit (e.g. Jamiat ul Burhani India)

jamaat
  id / jamaat_id VARCHAR(20) / jamiat_id FK → jamiat.id / name VARCHAR(200) / is_active
  -- Sub-unit; counselors and reviewers scoped to jamaat level
```

C. Case Lifecycle Tables

```
applicants
  id / its_number VARCHAR(8) UNIQUE / full_name / gender ENUM('male','female') / age INT
  phone / email / photo LONGTEXT / address TEXT
  jamiat_id FK / jamaat_id FK / jamiat_name / jamaat_name  -- denormalized for display

case_identifications
  id / its_number / full_name / age / gender / phone / email / photo / address
  jamiat VARCHAR(100) / jamaat VARCHAR(100)  -- free text; pre-formal Jamiat lookup
  eligible_in INT FK → case_types.id
  total_family_members INT / earning_family_members INT
  individual_income INT / family_income INT
  remarks TEXT / status ENUM('pending','eligible','ineligible')
  reviewed_by FK / reviewed_at / review_remarks
  case_id INT FK → cases.id  -- populated when marked eligible
  created_by FK / created_at / updated_at

case_identification_income_logs
  id / case_identification_id FK / changed_by FK / changed_at
  old_individual_income / new_individual_income
  old_family_income / new_family_income
  -- Immutable append-only audit log; never updated or deleted

cases
  id / case_number VARCHAR(50) UNIQUE  -- format: BS-0001, SH-0001
  applicant_id FK → applicants.id
  case_type_id FK → case_types.id
  jamiat_id FK / jamaat_id FK
  status VARCHAR(64)                  -- e.g. 'draft', 'counseling', 'welfare_review'
  status_id FK → statuses.id          -- normalized status ref
  current_workflow_stage_id FK → workflow_stages.id
  current_executive_level INT
  workflow_history LONGTEXT           -- JSON array of {stage, status, actor, ts} transitions
  roles INT FK → users.id             -- DCM assigned
  assigned_counselor_id FK → users.id
  is_duplicate TINYINT(1) DEFAULT 0
  priority ENUM('low','medium','high','urgent')
  description TEXT / notes TEXT
  created_by FK / created_at / updated_at
  [timestamp fields: registration_date, file_submission_date,
   executive_approval_date, manzoori_receipt_date,
   first_disbursement_date, completion_date]
```

D. Counseling Form Sub-Tables

The counseling form is decomposed into 7 independent tables linked via `counseling_forms` as the aggregate root. This allows section-by-section saving without locking or updating the entire form row.

```
counseling_forms                          -- aggregate root
  id / case_id FK UNIQUE / is_complete / completed_at
  personal_details_id FK → personal_details.id
  family_details_id FK → family_details.id
  assessment_id FK → assessment.id
  financial_assistance_id FK → financial_assistance.id
  economic_growth_id FK → economic_growth.id
  declaration_id FK → declaration.id
  attachments_id FK → attachments.id

personal_details      -- 1:1 case; ITS, name, age, education, contact, address
family_details        -- 1:1 case; income/expense breakdown, assets/liabilities, wellbeing text
  + family_members[] -- 1:M family_details; relation_id, education_id, occupation_id, annual_income
assessment            -- 1:1 case; background, proposed business, counselor evaluation; products[]
  + assessment_products_services[]
financial_assistance  -- 1:1 case; enayat/qardan totals, support_needed JSON, qh_fields JSON
  + financial_assistance_timeline[]        -- purpose + enayat + qardan + months
  + financial_assistance_timeline_assistance[] -- by period: immediate, after_1st_yr, etc.
  + financial_assistance_action_plan[]     -- sequential actions by timeline_period
  + financial_assistance_mentors[]         -- ITS, name, contact, email, photo
  + financial_assistance_qh_repayment_schedule[] -- QH name + year1-year5
economic_growth       -- 1:1 case; ~90 decimal columns for 5-year projections
  + economic_growth_projections[]          -- flexible category rows
declaration           -- 1:1 case; applicant + counselor + TR committee signatures
attachments           -- 1:1 case; boolean flags for document checklist
```

Design Note: Using 7 child tables (vs. a single wide counseling_forms table with 200+ columns) allows independent INSERT/UPDATE per section, clean FK constraints, and prevents MySQL row size limits. The aggregate root `counseling_forms` holds FKs to child records, created lazily on first section save.

E. Workflow & Approval Tables

```
workflow_stages
  id / stage_name / stage_key VARCHAR(50) UNIQUE
  description / sort_order INT / next_stage_id INT FK (self-ref)
  case_type_id FK → case_types.id  -- NULL = applies to all case types
  is_active / auto_advance_on_approve / requires_comments_on_reject
  associated_statuses LONGTEXT     -- JSON: ["welfare_review","pending_welfare"]
  sla_value DECIMAL(10,2) / sla_unit ENUM('hours','days','business_days','weeks','months')
  sla_warning_value / sla_warning_unit

workflow_stage_roles
  id / workflow_stage_id FK / role_id FK
  can_approve / can_reject / can_review / can_view
  can_edit / can_delete / can_create_case / can_fill_case  -- all TINYINT(1)

workflow_stage_users            -- individual user overrides at stage level
  id / workflow_stage_id FK / user_id FK
  [same permission flags as workflow_stage_roles]

status_history                  -- append-only; never updated
  id / case_id FK / from_status / to_status / changed_by FK / comments TEXT / created_at

workflow_comments
  id / case_id FK / user_id FK / workflow_step VARCHAR(50)
  comment TEXT / comment_type ENUM('general','note','feedback','approval','rejection')
```

F. Payment & Fund Utilization

```
payment_schedules
  id / case_id FK / payment_type ENUM('qardan_hasana','enayat')
  year_number INT (1-5) / disbursement_year YEAR / disbursement_month INT
  disbursement_amount DECIMAL(15,2) / repayment_months INT
  repayment_start_year / repayment_start_month
  is_disbursed TINYINT(1) / disbursed_date DATE / disbursed_by FK / disbursed_at
  created_by FK / updated_by FK / created_at / updated_at

payment_schedule_repayments
  id / payment_schedule_id FK / repayment_year YEAR
  repayment_month INT / repayment_amount DECIMAL(15,2)

fund_utilization_reports
  id / case_id FK / quarter ENUM('q1','q2','q3','q4') / year INT
  responses_json LONGTEXT  -- keyed by question_id: { "42": "answer text", "43": ["opt1"] }
  created_by FK / updated_by FK / created_at / updated_at

fund_utilization_sections
  id / title VARCHAR(200) / sort_order / is_active

fund_utilization_questions
  id / section_id FK / label VARCHAR(500) / field_type VARCHAR(50)
  options_json TEXT / min_length / max_length / max_file_size_mb / max_files
  table_config JSON / parent_question_id FK (self-ref) / parent_trigger_values TEXT
  placeholder / sort_order / is_required / is_auto_fill / auto_fill_key

fund_utilization_photos
  id / fund_utilization_id FK / file_path / file_name
  photo_type ENUM('shop_site','stock_equipment','applicant_work','other')

fund_utilization_files
  id / fund_utilization_id FK / question_id FK / file_path / file_name / file_size / mime_type
```

4.3 Index Strategy

TABLE	INDEX	RATIONALE
`cases`	`(status, jamiat_id, jamaat_id)`	Primary filter combination in case list queries
`cases`	`(assigned_counselor_id)`	Counselor dashboard case list
`cases`	`(current_workflow_stage_id)`	Stage-based filtering
`case_identifications`	`(its_number)`	Duplicate ITS check is O(1)
`applicants`	`(its_number)` UNIQUE	ITS lookup for case creation
`notifications`	`(user_id, is_read)`	Unread count query per user
`status_history`	`(case_id)`	Timeline fetch per case
`user_roles`	`(user_id, is_active)`	Permission resolution on every request
`fund_utilization_reports`	`(case_id, quarter, year)` UNIQUE	Prevents duplicate quarterly reports

5. Core Module Designs

5.1 Authentication & JWT Middleware

Authentication is fully stateless. No server-side session store. Every request must carry a valid JWT in the `Authorization: Bearer <token>` header.

```
// backend/middleware/auth.js — verifyToken()
const verifyToken = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'Access denied.' });

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);

    // Always re-fetch user to catch deactivation after token issue
    const [rows] = await pool.execute(
      'SELECT id, role, executive_level, is_active FROM users WHERE id = ?',
      [payload.id]
    );
    if (!rows[0] || !rows[0].is_active)
      return res.status(401).json({ error: 'User inactive or not found' });

    // Resolve all active roles + Jamiat/Jamaat scopes from user_roles junction
    const [roleRows] = await pool.execute(
      `SELECT r.name, ur.jamiat_ids, ur.jamaat_ids
         FROM user_roles ur JOIN roles r ON r.id = ur.role_id
        WHERE ur.user_id = ? AND ur.is_active = 1
          AND (ur.expires_at IS NULL OR ur.expires_at > NOW())`,
      [payload.id]
    );

    req.user = {
      ...rows[0],
      assignedRoles: roleRows.map(r => r.name),
      jamiats: parseIds(roleRows.flatMap(r => r.jamiat_ids)),
      jamaats: parseIds(roleRows.flatMap(r => r.jamaat_ids)),
    };
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid token' });
  }
};
```

Token issuance on login: `jwt.sign({ id, role }, JWT_SECRET, { expiresIn: JWT_EXPIRES_IN || '24h' })`. The token carries only minimal claims; full profile is fetched on each request to ensure deactivation takes effect immediately.

Password hashing paradigm: `bcrypt.hash(password, 10)` — cost factor 10 (~100ms hash time, sufficient for brute-force resistance). Legacy plain-text passwords in imported user data are detected and re-hashed on first successful login.

5.2 Organizational Scoping Engine

The scoping engine is the most critical security component. It ensures users can only read and modify data within their organizational boundaries.

```
// Simplified scope filter injection — applyScopeFilter()
// Called before route handlers on all case/applicant list endpoints

const BYPASS_ROLES = ['super_admin', 'admin'];
const APPROVE_SCOPE = 'jamiat';   // welfare, ZI, executive → jamiat-level scope
const FILL_SCOPE    = 'jamaat';   // counselors → jamaat-level scope

applyScopeFilter = (req, res, next) => {
  const { assignedRoles, jamiats, jamaats } = req.user;

  if (assignedRoles.some(r => BYPASS_ROLES.includes(r))) {
    req.scopeFilter = '';  // no WHERE restriction
    return next();
  }

  if (assignedRoles.some(r => APPROVE_ROLES.includes(r)) && jamiats.length) {
    req.scopeFilter = `AND c.jamiat_id IN (${jamiats.join(',')})`;
  } else if (assignedRoles.some(r => FILL_ROLES.includes(r)) && jamaats.length) {
    req.scopeFilter = `AND c.jamaat_id IN (${jamaats.join(',')})`;
  } else {
    // Counselor assigned directly to cases — no blanket scope, case-level check
    req.scopeFilter = `AND c.assigned_counselor_id = ${req.user.id}`;
  }
  next();
};
```

Security constraint: The `req.scopeFilter` fragment is only ever built from whitelisted integer arrays (parsed with `parseInt()` validation) — never from raw user input strings. SQL injection via scope manipulation is structurally impossible.

5.3 Case Identification Pipeline

Case identification is the entry point for all welfare cases. The pipeline involves ITS fetch, duplicate detection, income assessment, and conditional auto-case-creation on eligibility approval.

```
POST /api/case-identifications
  |
  ├─ 1. Joi: validate its_number(8 chars), income fields, eligible_in
  ├─ 2. Duplicate check:
  |      SELECT id, status FROM case_identifications
  |       WHERE its_number = ? AND status IN ('pending','eligible')
  |       LIMIT 1
  |      → 409 if found
  |
  ├─ 3. ITS auto-fetch (non-blocking attempt):
  |      GET external_ITS_API/contact/{its_number}
  |      → merge full_name, phone, email, photo into insert payload
  |      → catch: proceed with manually entered data only
  |
  ├─ 4. INSERT case_identifications (status='pending')
  └─ 5. 201 { id }

PUT /api/case-identifications/:id/review  (status → eligible)
  |
  ├─ 1. UPDATE case_identifications SET status='eligible', reviewed_by, reviewed_at
  ├─ 2. Upsert applicant:
  |      INSERT INTO applicants ... ON DUPLICATE KEY UPDATE full_name=VALUES(full_name)
  ├─ 3. Auto-create case:
  |      a. Generate case_number: BS-{LPAD(MAX(id)+1, 4, '0')}
  |      b. INSERT cases (status='draft', applicant_id, case_type_id, ...)
  |      c. UPDATE case_identifications SET case_id = lastInsertId
  ├─ 4. notificationService.create(DCM_users, caseId, 'New case created from identification')
  └─ 5. 200 { case_id, case_number }

PUT /api/case-identifications/:id/edit
  |
  ├─ 1. Check status IN ('pending','ineligible') — cannot edit eligible records
  ├─ 2. Capture old values from DB before UPDATE
  ├─ 3. UPDATE case_identifications SET income/family fields
  ├─ 4. INSERT case_identification_income_logs (old vs new) — ALWAYS; never skip
  └─ 5. 200
```

5.4 Case Management & Status Machine

Cases transition through a defined set of statuses driven by workflow actions. The status machine is enforced server-side — clients cannot set arbitrary status values directly.

ACTION ENDPOINT	FROM STATUS	TO STATUS	SIDE EFFECTS
POST /cases	—	draft	status_history INSERT, notify DCM
PUT /cases/:id/welfare-approve	welfare_review	zi_review	status_history, notify ZI users
PUT /cases/:id/welfare-reject	welfare_review	welfare_rework	status_history, comment required, notify DCM
PUT /cases/:id/welfare-forward-rework	welfare_review	dcm_rework	status_history, notify DCM
PUT /cases/:id/resubmit-welfare	dcm_rework / welfare_rework	welfare_review	status_history, notify welfare
PUT /cases/:id/zi-approve	zi_review	next stage key	Resolves next_stage_id from workflow_stages
PUT /cases/:id/zi-reject	zi_review	welfare_review	comment required, notify welfare
PUT /cases/:id/executive-approve	executive_L{N}	executive_L{N+1} or manzoori	Reads current_executive_level, increments or finalizes
PUT /cases/:id/executive-rework	executive_L{N}	welfare_review	Resets executive level; comment required

ACTION ENDPOINT	FROM STATUS	TO STATUS	SIDE EFFECTS
POST /cases/:id/close	any	closed	case_closures + case_closure_documents insert

All workflow action handlers share a common pattern:

```
// Shared workflow action pattern (simplified)
async function performWorkflowAction(caseId, toStatus, toStageId, actorId, comments) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();

    // 1. Update case status + stage atomically
    await conn.execute(
      `UPDATE cases SET status=?, current_workflow_stage_id=?,
        current_stage_entered_at=NOW(),
        workflow_history=JSON_ARRAY_APPEND(workflow_history,'$',JSON_OBJECT(
          'stage',?,'status',?,'actor',?,'ts',NOW()
        ))
        WHERE id=?`,
      [toStatus, toStageId, toStageId, toStatus, actorId, caseId]
    );

    // 2. Append to status_history (append-only audit)
    await conn.execute(
      `INSERT INTO status_history (case_id, from_status, to_status, changed_by, comments)
         VALUES (?,?,?,?,?)`,
      [caseId, currentStatus, toStatus, actorId, comments]
    );

    await conn.commit();

    // 3. Async side effects (non-blocking)
    notifyNextStageActors(caseId, toStageId);
  } catch (e) {
    await conn.rollback();
    throw e;
  } finally { conn.release(); }
}
```

Transaction pattern: Status updates + audit log inserts always run inside a single DB transaction. A partial write (status updated but history not written) is impossible.

5.5 Counseling Form Architecture

The counseling form is the most complex data entry surface in the system. It uses an aggregate root pattern with lazy child record creation.

```
GET /api/counseling-forms/case/:caseId
  |
  ├─ 1. SELECT counseling_forms WHERE case_id = ? (with all FK IDs)
  ├─ 2. IF NOT EXISTS → INSERT counseling_forms (case_id, is_complete=0) auto-create shell
  ├─ 3. For each section FK that is NOT NULL → JOIN fetch that section's data
  ├─ 4. For each section FK that IS NULL → return empty object for that section
  ├─ 5. Attach stage_permissions for req.user.role (from counseling_form_stage_permissions)
  └─ 6. Return merged form object + stage_permissions

PUT /api/counseling-forms/:formId/section/:section
  |
  ├─ section ∈ { personal_details, family_details, assessment,
  |               financial_assistance, economic_growth, declaration, attachments }
  |
  ├─ Route to section handler (e.g. updateFamilyDetails()):
  |
  |  family_details example:
  |  ├─ Joi validate section payload
  |  ├─ conn.beginTransaction()
  |  ├─ UPSERT family_details (INSERT ... ON DUPLICATE KEY UPDATE ...)
  |  ├─ DELETE family_members WHERE family_details_id = ?   -- full replace pattern
  |  |    then bulk INSERT new family_members rows
  |  ├─ UPDATE counseling_forms SET family_details_id = ? WHERE id = ?
  |  ├─ conn.commit()
  |  └─ 200 { message, section, updated: true }
  |
  └─ financial_assistance example (complex sub-tables):
     ├─ UPSERT financial_assistance core row
     ├─ DELETE + bulk INSERT: timeline, timeline_assistance, action_plan, mentors, qh_schedule
     └─ UPDATE counseling_forms FK
```

Full-replace pattern for sub-arrays: DELETE all child rows for the parent, then bulk INSERT the submitted array. Simpler than differential merge; safe because the entire section is submitted together. Transaction wraps both operations atomically.

5.6 Workflow & Approval Engine

Workflow stages are data-driven — stored in `workflow_stages` and resolved at runtime. No hardcoded stage sequences in application code.

```
Stage resolution algorithm (PUT /cases/:id/workflow-action — generic):

1. FETCH current stage: workflow_stages WHERE id = case.current_workflow_stage_id
2. VALIDATE actor permissions:
   a. CHECK workflow_stage_users WHERE workflow_stage_id=? AND user_id=req.user.id
   b. OR CHECK workflow_stage_roles WHERE workflow_stage_id=? AND role_id IN (user's roles)
   → action='approve' requires can_approve=1; action='reject' requires can_reject=1
3. IF action='approve':
   a. IF stage.auto_advance_on_approve=1:
      → next_stage_id = stage.next_stage_id
      → resolve next stage's associated_statuses[0] as new status
   b. ELSE → use explicitly passed next_stage_id from request
4. Validate: stage.requires_comments_on_reject → 400 if empty on reject
5. performWorkflowAction(caseId, newStatus, newStageId, actorId, comments)
6. notifyStageActors(newStageId, caseId)  -- async

Executive level resolution (special case):
  current_executive_level = case.current_executive_level
  → IF next level exists in executive_levels (ordered by level_number):
       advance to executive_L{N+1} stage
     ELSE:
       advance to manzoori stage (final sanction)
  → UPDATE cases.current_executive_level = nextLevel || NULL
```

5.7 SLA Calculation Engine

SLA is calculated dynamically — not stored as a computed field — to always reflect current configuration changes.

```
// utils/slaCalculator.js
function calculateSlaStatus(stageEnteredAt, slaValue, slaUnit,
                            slaWarningValue, slaWarningUnit) {
  const now = new Date();
  const entered = new Date(stageEnteredAt);

  const elapsedMs = now - entered;
  const breachMs  = toMs(slaValue, slaUnit);
  const warnMs    = breachMs - toMs(slaWarningValue, slaWarningUnit);

  if (elapsedMs >= breachMs)  return { status: 'breached', breachedAt: new Date(entered.getTime()+breachMs) };
  if (elapsedMs >= warnMs)    return { status: 'at_risk'  };
  return { status: 'on_time' };
}

// toMs() handles: hours, days, business_days (excludes weekends), weeks, months
// Business days calculation iterates calendar days, skipping Sat/Sun
```

SLA info is appended to every case in the list response as `slaInfo: { status, breachedAt? }`. The notification service's `POST /notifications/check-sla` endpoint iterates all open cases, recalculates SLA, and creates warning/breach notifications in bulk.

5.8 Fund Utilization Dynamic Form Builder

The fund utilization form is entirely data-driven. Admins configure sections and questions via the master API; the frontend renders them generically.

```
GET /api/fund-utilization-master/form-config
  → Returns full tree: sections[] each with questions[]
  → question.field_type ∈ { text, number, textarea, select, multiselect,
                            radio, checkbox, date, file, table }
  → question.parent_question_id + parent_trigger_values → conditional logic
  → question.is_auto_fill + auto_fill_key → pre-populate from case data

Frontend rendering logic:
  sections.forEach(section →
    questions.forEach(q →
      IF q.parent_question_id:
        check current responses[parent_id] ∈ q.parent_trigger_values
        → render only if condition met
      ELSE: always render
      SWITCH q.field_type → render appropriate input component
      IF q.is_auto_fill → pre-fill from caseData[q.auto_fill_key]
    )
  )

POST /api/cases/:caseId/fund-utilization
  payload: { quarter, year, responses: { "questionId": value } }
  → Joi validate quarter ∈ ['q1','q2','q3','q4'], year INT
  → Check unique: SELECT id WHERE case_id=? AND quarter=? AND year=? → 409 if exists
  → INSERT fund_utilization_reports (responses_json = JSON.stringify(responses))
  → 201 { id }
```

5.9 PDF Generation Pipeline

Two PDF generation strategies are used based on document complexity.

Strategy A: PDFKit (Quarterly Reports)

```
utils/quarterlyReportPdfKit.js:
  1. Fetch report data: aggregateReportData(caseId, reportId)
     → JOIN: case, applicant, fund_utilization_reports, photos, files
  2. Build PDFDocument (pdfkit):
     doc.font('Helvetica').fontSize(10)
     doc.text('Q1 2025 Reassessment Report')
     → iterate sections → render labels + responses
     → embed photos (doc.image(photoBuffer, x, y, { width: 180 }))
  3. Pipe to response stream:
     res.setHeader('Content-Type', 'application/pdf')
     res.setHeader('Content-Disposition', 'attachment; filename="Q1_2025_Case-BS-0012.pdf"')
     doc.pipe(res)
     doc.end()
```

Strategy B: Puppeteer (Cover Letters)

```
routes/coverLetters.js → generateCoverLetter():
  1. Fetch cover_letter_form data + applicant + counselor info
  2. templates/quarterlyReportPdfTemplate.js → renderHtml(data)
     → returns styled HTML string with inline CSS (Puppeteer-safe)
  3. const browser = await puppeteer.launch({ headless: 'new', args: ['--no-sandbox'] })
     const page = await browser.newPage()
     await page.setContent(html, { waitUntil: 'networkidle0' })
     const pdfBuffer = await page.pdf({ format: 'A4', printBackground: true, margin: {...} })
     await browser.close()
  4. INSERT cover_letters (case_id, file_path, generated_by)
     → Write pdfBuffer to uploads/cover_letters/{caseId}_{timestamp}.pdf
  5. 200 { coverLetterId, filePath }
```

> **Puppeteer deployment:** Requires Chromium installed on server. Launch args `--no-sandbox --disable-setuid-sandbox` required in containerized/non-root Linux environments. PDF generation is CPU-intensive (~1–3s per document) — run synchronously per request, not in worker threads.

5.10 ITS External API Integration

The ITS (Institutional Information System) is an external API providing applicant identity data. Integration is designed to fail gracefully.

```
GET /api/applicants/fetch-from-api/:itsNumber

const fetchFromIts = async (itsNumber) => {
  try {
    const res = await axios.get(`${ITS_API_BASE}/contact/${itsNumber}`, {
      headers: { Authorization: `Bearer ${ITS_API_KEY}` },
      timeout: 5000  // 5s max; non-blocking for UI
    });
    return { success: true, data: normalizeItsResponse(res.data) };
  } catch (err) {
    // Log for monitoring; surface 'not_found' vs 'api_error' to caller
    if (err.response?.status === 404)
      return { success: false, reason: 'not_found' };
    return { success: false, reason: 'api_error', message: err.message };
  }
};

// normalizeItsResponse() maps ITS API field names → FACTS schema fields
// e.g. ITS 'ContactNumber' → 'phone', 'JamaatName' → 'jamaat_name'

Lookup strategy (GET /api/applicants/lookup/:itsNumber):
  1. SELECT * FROM applicants WHERE its_number = ?
     → IF found in local DB: return { source: 'database', data }
  2. ELSE: fetchFromIts(itsNumber)
     → IF success: return { source: 'api', data }
  3. ELSE: return { success: false }  → frontend shows manual entry form
```

5.11 Notification Service

```
// services/notificationService.js
const createNotification = async (userIds, caseId, title, message, type='info') => {
  if (!userIds?.length) return;
  const placeholders = userIds.map(() => '(?,?,?,?,?,0)').join(',');
  const values = userIds.flatMap(id => [id, caseId, title, message, type]);
  await pool.execute(
    `INSERT INTO notifications (user_id,case_id,title,message,type,is_read) VALUES ${placeholders}`,
    values
  );
  // Bulk INSERT — one query for N recipients (prevents N+1 for stage with 10+ users)
};

// SLA check (POST /api/notifications/check-sla)
async function runSlaCheck() {
  const openCases = await fetchOpenCasesWithStageInfo();
  for (const c of openCases) {
    const sla = calculateSlaStatus(c.stage_entered_at, c.sla_value, c.sla_unit,
                                   c.sla_warning_value, c.sla_warning_unit);
    if (sla.status === 'at_risk' || sla.status === 'breached') {
      const actors = await getStageActorIds(c.current_workflow_stage_id);
      await createNotification(actors, c.id,
        `SLA ${sla.status} — Case ${c.case_number}`,
        `Case has been in "${c.stage_name}" for ${c.elapsed} ${c.sla_unit}.`,
        sla.status === 'breached' ? 'error' : 'warning'
      );
    }
  }
}
```

5.12 File Upload & Attachment Handler

```
Multer configuration (routes/attachments.js):
  storage: multer.diskStorage({
    destination: (req, file, cb) => {
      const dir = path.join(UPLOAD_PATH, 'cases', req.params.caseId);
      fs.mkdirSync(dir, { recursive: true });  // auto-create case subdirectory
      cb(null, dir);
    },
    filename: (req, file, cb) => {
      const ext  = path.extname(file.originalname).toLowerCase();
      const safe = sanitize(file.originalname);  // strip path traversal chars
      cb(null, `${Date.now()}_${safe}`);
    }
  }),
  fileFilter: (req, file, cb) => {
    const ALLOWED = ['.pdf','.doc','.docx','.xls','.xlsx','.jpg','.jpeg','.png'];
    cb(null, ALLOWED.includes(path.extname(file.originalname).toLowerCase()));
  },
  limits: { fileSize: parseInt(MAX_FILE_SIZE) || 10_485_760 }  // 10MB default

Multi-upload route (POST /upload-multiple/:caseId):
  multer().array('files', 10)  // max 10 files per request
  → iterate req.files → build case_attachments insert array
  → bulk INSERT case_attachments
  → 200 { files: [{ attachmentId, fileName, fileSize, filePath }] }

Download route (GET /download/:attachmentId):
  → SELECT file_path, file_name WHERE id=? (auth check included)
  → res.download(absolutePath, fileName)  // streams file from disk
  → File never exposed via public static URL — always via authenticated endpoint
```

---

6. API Design Patterns

6.1 Route Structure Convention

CONVENTION	IMPLEMENTATION
Resource naming	Plural nouns: `/cases`, `/users`, `/applicants`, `/workflow-stages`
Nested resources	`/cases/:caseId/comments`, `/cases/:caseId/fund-utilization`
Actions (non-CRUD)	Verb suffixes: `/cases/:id/welfare-approve`, `/cases/:id/close`
Pagination	Query params: `?page=1&limit=20`. Response: `{ data[], total, page, limit, totalPages }`
Filtering	Query params matching column names: `?status=draft&jamiat_id=3`
Search	`?search=term` → LIKE on relevant columns (full_name, its_number, case_number)
Sorting	Default: `ORDER BY created_at DESC`. No client-controlled sort in v1.

6.2 Input Validation Pattern

```
// Every POST/PUT route validates with Joi before any DB operation
const schema = Joi.object({
  its_number:             Joi.string().length(8).required(),
  eligible_in:            Joi.number().integer().positive().required(),
  total_family_members:   Joi.number().integer().min(0).max(25).required(),
  individual_income:      Joi.number().min(0).max(9999999).required(),
});
const { error, value } = schema.validate(req.body, { abortEarly: false });
if (error) return res.status(400).json({ error: error.details.map(d => d.message) });
```

6.3 Error Response Standardization

HTTP STATUS	MEANING	BODY
200 OK	Success	`{ message, data? }` or resource object
201 Created	Resource created	`{ message, id, ... }`
400 Bad Request	Validation error	`{ error: "string or [array of messages]" }`
401 Unauthorized	No/invalid token or inactive user	`{ error: "Access denied." }`
403 Forbidden	Token valid, permission missing	`{ error: "Insufficient permissions" }`
404 Not Found	Resource doesn't exist or out of scope	`{ error: "Not found" }`
409 Conflict	Duplicate record constraint	`{ error: "Duplicate ITS / Case already exists" }`
500 Server Error	Unhandled exception	`{ error: "Internal server error" }` (no stack trace)

7. Frontend Architecture

7.1 State Management Strategy

STATE TYPE	TOOL	PATTERN
Server data (API responses)	React Query	`useQuery(key, fetchFn)` — cached, auto-refetched on window focus
Mutations (POST/PUT/DELETE)	React Query	`useMutation(mutateFn, { onSuccess: () => queryClient.invalidateQueries(key) })`
Authentication state	React Context	`AuthContext` : `{ user, token, login(), logout() }` — token stored in-memory (not localStorage)
Form state	React Hook Form	`useForm({ defaultValues })` ; `reset(data)` on fetch; `handleSubmit()` on save
Local UI state	useState	Modal open/close, pagination page, selected filters — never lifted to Context

7.2 Route Protection Pattern

```
// App.js — Multi-strategy route protection
<Route path="/cases" element={
  <ProtectedRoute requiredPermission={{ resource: 'cases', action: 'read' }}>
    <Cases />
  </ProtectedRoute>
} />

<Route path="/roles" element={
  <ProtectedRoute requireAdminAccess>
    <RoleManagement />
  </ProtectedRoute>
} />

// ProtectedRoute checks: auth → permission/role → render or redirect
// Permission check: user.permissions[resource]?.includes(action)
// Permissions resolved on login; attached to user object in AuthContext
```

7.3 Counseling Form Tab Architecture

```
CounselingForm.js:
  ├── 7 tab buttons (Personal | Family | Assessment | Financial | Economic | Declaration | Attachments)
  ├── useQuery('counselingForm', () => api.getCounselingForm(caseId))
  │    → populates all section data on mount
  │
  ├── Each tab renders its own sub-component:
  │   PersonalDetailsTab / FamilyDetailsTab / AssessmentTab / etc.
  │   ├── useForm({ defaultValues: formData.sectionData })
  │   ├── reset(formData.sectionData) on data load
  │   ├── handleSubmit → PUT /counseling-forms/:id/section/:section
  │   └── useMutation → invalidate ['counselingForm', caseId]
  │
  ├── Completion indicator: formData.is_complete + per-section derived flags
  └── Stage permission gates: stagePermissions[currentUserRole]?.can_update
       → disabled inputs if can_update=false for current user's role
```

7.4 SignaturePad Component

```
SignaturePad.js:
  Props: { value, onChange, label }

  Modes:
  ├── 'draw' mode:
  │   → <canvas> element with mouse/touch event listeners
  │   → canvas.toDataURL('image/png') → base64 string → onChange(base64)
  │   → Clear button: ctx.clearRect(0,0,width,height)
  │
  └── 'upload' mode:
      → <input type="file" accept="image/*,.pdf">
      → FileReader.readAsDataURL() → base64 → onChange(base64)

  Stored as: signature_type ENUM('draw','upload') + signature_file_path TEXT (base64)
  Rendered in PDF: Puppeteer renders the base64 img src inline
```

7.5 usePermission Hook

```
// frontend/src/utils/permissionUtils.js
export const usePermission = (resource, action) => {
  const { user } = useAuth();
  return user?.permissions?.[resource]?.includes(action) ?? false;
};

// Usage in components:
const canApprove = usePermission('cases', 'approve');
return canApprove ? <ApproveButton /> : null;
```

---

8. Security Architecture

8.1 Transport Security

- **HTTPS enforced:** Nginx terminates TLS; HTTP → HTTPS redirect. HSTS header set via Helmet.
- **CORS:** `Access-Control-Allow-Origin: FRONTEND_URL` — no wildcard in production.
- **CSP:** Helmet's `contentSecurityPolicy()` restricts script-src, img-src to known origins.

8.2 SQL Injection Prevention

```
// Absolute rule: ALL user-controlled values go through bind parameters
// CORRECT — parameterized:
const [rows] = await pool.execute(
  'SELECT * FROM cases WHERE id = ? AND jamiat_id = ?',
  [(int)req.params.id, (int)req.user.jamiats[0]]
);

// FORBIDDEN — never construct WHERE clauses from string concatenation
// EXCEPTION: scope filter fragments are built from whitelisted integer arrays only:
const safe = jamiats.map(id => parseInt(id)).filter(n => !isNaN(n));
const scopeSql = safe.length ? `AND jamiat_id IN (${safe.join(',')})` : '';
```

8.3 Authorization Matrix

RESOURCE	ADMIN	DCM	COUNSELOR	WELFARE	ZI	EXECUTIVE	FINANCE
cases:create	✓	✓	—	—	—	—	—
cases:read	✓ (all)	✓ (Jamiat)	✓ (assigned)	✓ (Jamiat)	✓ (Jamiat)	✓ (level)	✓ (stage)
cases:approve	✓	—	—	✓	✓	✓	—
cases:close_case	✓	—	—	—	—	—	—
counseling_forms:fill	✓	✓	✓	r/o	r/o	r/o	—
payment_management	✓	—	—	—	—	—	✓
fund_utilization	✓	—	—	—	—	—	✓
users:create/update	✓	—	—	—	—	—	—
master:update	✓	—	—	—	—	—	—

8.4 File Security Controls

CONTROL	IMPLEMENTATION
Extension whitelist	Multer fileFilter rejects any extension not in ALLOWED_EXTENSIONS array
MIME validation	file-type library checks magic bytes on upload (not just extension)
Path traversal prevention	`path.basename()` + sanitize-filename on stored filenames; case-isolated subdirectories
No public URL exposure	`uploads/` directory not served as Express static; all downloads via authenticated endpoint
Size limits	Multer `limits.fileSize` + Nginx `client_max_body_size 10m`

9. Performance & Scalability

9.1 Database Query Optimisation

- **Composite indexes** on all high-cardinality filter combinations (status + jamiat_id, case_id + quarter + year).
- **SELECT specificity:** All queries name explicit columns. No `SELECT *` in production code — avoids fetching 100KB photo blobs in list queries where only IDs and names are needed.
- **Pagination enforcement:** All list endpoints require LIMIT/OFFSET. Maximum `limit=500` hard cap. No unbounded result sets.
- **N+1 prevention:** Notification bulk INSERT; case list uses JOINs not per-row sub-selects; family members fetched in one `WHERE family_details_id IN (?)`.
- **Connection pooling:** mysql2 `createPool({ connectionLimit: 10 })` handles burst concurrency without connection overhead per request.

9.2 Frontend Performance

- **React Query caching:** Case list cached for 5 minutes by default; stale-while-revalidate prevents empty loading states on navigation.
- **Code splitting:** React Router lazy-loads page components — initial bundle does not include PDF viewer, fund utilization form builder, or role management pages.
- **Photo handling:** Applicant/user photos stored as base64 in DB. List queries join only id/name — photos fetched only on detail view to prevent bloated list responses.
- **Excel import chunking:** Server-side xlsx parsing uses streaming mode (`readFile` with row-by-row processing) — prevents memory spikes for 500+ row imports.

9.3 Scaling Ceiling & Upgrade Path

BOTTLENECK	CURRENT LIMIT	UPGRADE PATH
Concurrent users	~50 (single Node.js process)	PM2 cluster mode (N = CPU cores); Nginx upstream load balancing
File storage	Server disk (EBS volume)	AWS S3 + signed URL downloads; Multer S3 storage engine
PDF generation	~3s/doc (Puppeteer)	Worker thread pool; queue-based generation (Bull/BullMQ)
DB connections	Pool: 10	RDS read replica for reporting queries; increase pool size

---

10. Deployment & Infrastructure

10.1 Production Topology

```
[ Internet ]
     │  HTTPS (443)
     ▼
[ Nginx 1.24 ]
  ├── SSL termination (Let's Encrypt / custom cert)
  ├── HTTP → HTTPS redirect
  ├── location / → serve React build (frontend/build/)
  ├── location /api → proxy_pass http://localhost:5000
  └── location /uploads → BLOCKED (never serve directly)
     │
     ▼
[ Node.js Express — PM2 managed ]
  Process: facts-backend
  Port: 5000
  PM2 config: { instances: 1, watch: false, restart_delay: 3000 }
     │
     ▼
[ MySQL 8.0 ]
  Database: baaseteen_cms
  Bind: 127.0.0.1 only (no external DB access)
  Root password: env-managed
     │
     ▼
[ Local Filesystem ]
  /var/www/FACTS/backend/uploads/   (write access: www-data / node process user)
  /var/log/facts_error.log          (append-only; logrotate configured)
```

10.2 Nginx Site Configuration

```
server {
    listen 443 ssl http2;
    server_name facts.yourdomain.com;
    ssl_certificate     /etc/ssl/facts/cert.crt;
    ssl_certificate_key /etc/ssl/facts/key.key;

    root /var/www/FACTS/frontend/build;
    index index.html;

    # React SPA — all unknown routes serve index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # API reverse proxy
    location /api {
        proxy_pass         http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header   Upgrade $http_upgrade;
        proxy_set_header   Connection 'upgrade';
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
        proxy_set_header   X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header   X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;          # allow for PDF generation
    }

    client_max_body_size 10M;            # matches Multer file size limit
}

server {
    listen 80;
    server_name facts.yourdomain.com;
    return 301 https://$server_name$request_uri;
}
```

10.3 PM2 Ecosystem Configuration

```
// ecosystem.config.js
module.exports = {
  apps: [{
    name: 'facts-backend',
    script: './backend/index.js',
    instances: 1,
    autorestart: true,
    watch: false,
    max_memory_restart: '512M',
    env_production: {
      NODE_ENV: 'production',
      PORT: 5000,
    },
    error_file: '/var/log/pm2/facts-error.log',
    out_file:   '/var/log/pm2/facts-out.log',
    log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
  }]
};
```

10.4 Database Backup Strategy

```
# Daily automated backup cron (crontab -e)
0 2 * * * mysqldump -u root -p${DB_PASS} baaseteen_cms \
  | gzip > /backups/facts_$(date +\%Y\%m\%d).sql.gz

# Keep last 30 days
0 3 * * * find /backups/ -name "facts_*.sql.gz" -mtime +30 -delete

# Migration runner
node backend/scripts/run-migration.js database/migrations/new_migration.sql
```

10.5 Post-Deployment Checklist

STEP	COMMAND / ACTION
Database import	`mysql -u root -p baaseteen_cms < database/baaseteen_production_database.sql`
Dependencies install	`cd backend && npm ci --omit=dev`
Frontend build	`cd frontend && npm ci && npm run build`
Environment file	Copy `.env` with production DB credentials + strong JWT_SECRET (min 32 chars)
Upload dir permissions	`mkdir -p backend/uploads && chown -R www-data:www-data backend/uploads`
Chromium (Puppeteer)	`npx puppeteer browsers install chrome`
Start backend	`pm2 start ecosystem.config.js --env production && pm2 save`
Configure Nginx	Copy site config; `nginx -t && systemctl reload nginx`
Change default password	Login as admin / password → change immediately via /profile
Verify health	`curl https://your-domain/api/health` → `{ status: "ok" }`

---

11. Error Handling & Logging

11.1 Global Error Handler

```
// backend/index.js — last middleware registered
app.use((err, req, res, next) => {
  const requestId = req.headers['x-request-id'] || uuidv4();

  // Structured log: never expose stack trace to client
  console.error(JSON.stringify({
    level: 'error',
    requestId,
    method: req.method, path: req.path,
    userId: req.user?.id,
    error: err.message,
    stack: err.stack,
    ts: new Date().toISOString()
  }));

  // Multer-specific error handling
  if (err.code === 'LIMIT_FILE_SIZE')
    return res.status(400).json({ error: 'File too large. Maximum 10MB per file.' });
  if (err.code === 'LIMIT_FILE_COUNT')
    return res.status(400).json({ error: 'Too many files. Maximum 10 per upload.' });

  // DB constraint violations
  if (err.code === 'ER_DUP_ENTRY')
    return res.status(409).json({ error: 'Duplicate entry — record already exists.' });

  // Safe generic fallback
  res.status(500).json({ error: 'Internal server error', requestId });
});
```

11.2 Validation Error Pattern (Frontend)

```
// Axios interceptor — frontend/src/utils/api.js
api.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) {
      authContext.logout();        // clear token, redirect to login
    }
    if (error.response?.status === 403) {
      toast.error('Access denied. Insufficient permissions.');
    }
    return Promise.reject(error);
  }
);
```

11.3 Log Structure

LOG LEVEL	EVENTS	DESTINATION
ERROR	Unhandled exceptions, DB errors, external API failures, file write failures	PM2 error log + `/var/log/pm2/facts-error.log`
WARN	JWT near-expiry, SLA breaches, ITS API fallback triggered	PM2 out log
INFO	Server start, DB connection pool ready, migration applied	PM2 out log
DEBUG	Query plans, scope filters applied (NODE_ENV=development only)	Console (not in production)

No sensitive data in logs: Passwords, JWT tokens, ITS API keys, and full file paths are never logged. User IDs and case numbers are included for traceability. All log output is structured JSON for parsing by log aggregators (e.g. CloudWatch, Datadog).

— End of Technical Design Document —

FACTS — File and Case Tracking System
Version 3.7 — April 23, 2025